

THE OPTIONS (SHORTCALL) DECLARATION

The OPTIONS(SHORTCALL) declaration is useful for calling PMA procedures with the PMA instruction JSXB instead of the more common PCL instruction. A procedure call of this type is faster than one using PCL. However, the called procedure must be written to expect this kind of call. In Rev. 18 and Rev. 19, the only system subroutine that can (and must) be declared in this way is MKONU\$.

The format of this declaration is:

```
DECLARE procedure-name ENTRY OPTIONS(SHORTCALL [stack-size] );
```

stack-size specifies the extra space needed for the calling procedure's stack. The default size is 8.

The call does not generate a new stack for storage, as does PCL. The calling procedure's stack space is used. Thus it may be necessary to specify stack size in the declaration in order to enlarge the calling stack. For example, MKONU\$ requires an 28-word stack, so the user's stack must be large enough to accomodate this requirement. If stack size is not large enough, the return from the subroutine will cause unpredictable error messages.

Arguments may be used with the SHORTCALL option. The computer will set up the L register to point to a vector containing the addresses of the arguments, or, in the case of one argument, to the address of the argument itself. No type checking is done. For Rev. 19, there are no standard subroutine calls that require both SHORTCALL and argument passing.

DATA TYPES

Table 7-1 summarizes the argument types of FORTRAN subroutines and functions that can be called from PL1G. The following is a discussion of these argument types, as well as some generic types, and how they relate to PL1G data types and structures. The PL1G CHAR(*)VARYING argument type is discussed briefly.

INTEGER*2

The INTEGER*2 expected by FORTRAN subroutines is PL1G's FIXED BIN, also called FIXED BIN(15). Sample Program 1 includes a call to the subroutine SRCH\$\$, which expects an INTEGER*2 argument.

Notes to Table 7-1

- (1) If used for representing true (1) and false (0), negative numbers are true, positive numbers and 0 are false. This is not compatible with FORTRAN. In PL1G, '1'B is true; if this value is stored in a 16-bit integer, the sign bit is set, giving 100000 octal, or -32768 decimal. False in PL1G may always be represented as decimal 0.
- (2) LOGICAL data in FORTRAN represents true and false as 1 and 0, respectively. This is not directly compatible with Pascal or PL1G.
- (3) Boolean data in Pascal is represented in 16 bits where the sign bit determines true and false. (A negative sign means true, a positive sign means false.) This data type is directly compatible with a BIT(1) ALIGNED variable in PL1G.
- (4) To define a 32-bit integer in Pascal, use an integer array whose positive limit is greater than 32768 and whose negative limit is less than -32768.
- (5) Where "n" is a constant expression with the program module. This is not a dynamic length.
- (6) A character-varying string can be simulated in each language indicated, as discussed in the chapter on that language.
- (7) This implementation of a pointer in PL1G is subject to change; a program that passes pointers or receives them may have to be recompiled, and a program that assumes a particular form or size of pointer data may have to be rewritten.
- (8) Where <type> is either a user-defined type or a standard Pascal type.

Sample Program 4 calls the function `DELE$A`, which returns a logical value.

CHARACTER(*)VARYING

This argument is expected only by PL1G subroutines. It should be declared as `CHAR(n)VARYING`, and passed only as a data name, not as a literal. No other special steps are needed to pass `CHAR(*)VARYING` from a PL1G program.

Sample Program 5 calls the PL1G subroutine `GV$GET`, which expects a `CHAR(*)VARYING` argument.

CHARACTER(n)NONVARYING, POINTER, BIT(1)

These arguments are expected only by PL1G standard subroutines. They should be declared the same way in the calling routine.

USING SYSCOM TABLES

Subroutine descriptions in this guide sometimes refer to codes with names in the format `x$yyyy`, where `x` and `y` are letters. The code names may be used in the program instead of numeric values. There are three groups of these codes.

Error codes have names in the format `E$yyyy`. These equivalents should be inserted in the PL1G program before the subroutine declaration with the statement:

```
%INCLUDE 'SYSCOM>ERRD.INS.PL1';
```

The equivalents for these key codes are in Appendix D and in the file `SYSCOM>ERRD.INS.PL1`.

Key codes have names in the format `K$yyyy`. These equivalents should be inserted in the program with the statement:

```
%INCLUDE 'SYSCOM>KEYS.INS.PL1';
```

The equivalents for these key codes are in Appendix C and in the file `SYSCOM>KEYS.INS.PL1`.

Some subroutines in VAPPLB use argument codes in the form `A$yyyy`. These codes should also be inserted with the statement `%INCLUDE 'SYSCOM>A$KEYS.INS.PL1'`. They are listed in the table at the end of Chapter 12 on VAPPLB, or in the file `SYSCOM>A$KEYS.INS.PL1`.